# EECE503N / EECE798N: Final Project

AI engineering production system

# LOCUS

**Team Members:**
Marc El Nawar
Tatiana Nohra

May 3, 2026

# Contents

# 1 Project overview and positioning

## 1.1 Problem definition

The core operational problem addressed by locus is the inefficiency and friction inherent in both traditional physical retail and modern e-commerce. Physical shopping often requires consumers to spend countless hours commuting and searching multiple stores without a guarantee of finding a specific desired item. Conversely, while online shopping offers high searchability, it introduces issues such as shipping delays and the inability to physically inspect or try on garments, which frequently leads to unsatisfactory quality upon delivery.

Locus resolves these tradeoffs by allowing users to upload a reference photo of an outfit to identify visually matching clothing pieces available within a user-specified geographical radius. This system optimizes the shopping experience by:

- **Minimizing search and commute time:** Users are directed precisely to nearby stores that stock their desired items based on how far they are willing to travel, transforming physical shopping from a hassle into an efficient, targeted trip.
- **Ensuring product satisfaction:** By facilitating in-store try-ons, locus eliminates the quality uncertainty and shipping delays associated with online retail while preserving the physical shopping experience.
- **Empowering local businesses:** The platform acts as a discovery engine, providing crucial visibility to local shop owners by directly connecting them with high-intent nearby shoppers.

## 1.2 Target audience and novelty

The primary business audience comprises small, independent retail merchants operating Shopify-based platforms. These businesses often lack the brand recognition required to drive organic traffic, despite stocking highly desirable inventory. For the consumer side, the audience consists of high-intent shoppers seeking specific stylistic items rather than generic commodities.

The system incentivizes merchants by acting as a hyper-local discovery engine, providing direct visibility for their items based on visual merit rather than brand power. Our explicit novelty claim lies in the geographical constraint applied to the visual search. While existing tools like Google Lens provide globalized e-commerce links, locus introduces a strict, user-defined radius filter. This transforms visual search from a broad internet query into an actionable, local retail strategy, making it highly relevant for immediate physical acquisition within the user's context.

## 1.3 Non-AI baseline justification

The non-AI baseline for this system is a traditional, text-based query search application relying on keyword matching and manual metadata tags.

This baseline is inherently insufficient due to the semantic gap between text descriptions and complex visual clothing patterns. A text-based search loses critical specificity, making it exceptionally difficult to retrieve exact visual matches for unique or highly stylized garments. Since locus targets users seeking specific aesthetics—rather than easily searchable commodity items like a "plain white t-shirt"—the text baseline invalidates the core product value. Therefore, an AI-driven approach utilizing image embeddings and visual similarity search is strictly necessary to capture and match the visual nuance required by our target audience.loy or pay for this system and state your novelty claim explicitly.

# 2 System architecture

## 2.1 Service boundaries

## 2.2 Endpoint specifications

- **External endpoint (EEP):** The External Endpoint is built with FastAPI and acts as the primary system gateway. It serves the frontend application directly from the virtual machine while orchestrating all client-to-backend requests.
  - **Frontend Serving & Authentication:** The EEP mounts and serves the static Single Page Application (SPA) assets directly to the user[cite: 16]. It includes a dedicated `/auth` router that manages secure merchant access using JSON Web Tokens (JWT) and bcrypt password hashing.
  - **Input Validation & Constraints:** To guarantee system stability and prevent memory exhaustion, a custom middleware explicitly caps all HTTP payloads at a 10 MB maximum upload limit[cite: 16]. The `slowapi` library is integrated to enforce strict IP-based rate limiting, restricting users to 10 searches and 5 detection queries per minute.
  - **Routing & Orchestration Logic:** Upon receiving an image query, the EEP applies bounding box crops and routes the payload to the internal visual engine for vectorization[cite: 16]. The EEP implements dynamic query routing; for example, if the query involves accessories (shoes, bags, hats), it automatically injects style-specific text hints or drops strict category filters to compensate for known model biases.
  - **Asynchronous Background Processing:** To ensure a low-latency user experience, the EEP heavily utilizes FastAPI background tasks. Immediately following a search, it spawns non-blocking workers to extract visual attributes and triggers an LLM judge (Gemini 2.0 Flash via OpenRouter) to evaluate and score the visual similarity of the top results.
  - **Observability Integration:** The gateway is instrumented with Prometheus, continuously exposing custom metrics such as `locus_searches_total`, feedback star distributions, and real-time active session counts.

# 3 Project overview and positioning

## 3.1 Containerization and deployment

[Detail the Docker configuration for all three images and the Kubernetes/Docker Compose setup used for cloud deployment][cite: 1].

# 4 Engineering tradeoffs

[Identify at least 3 engineering tradeoffs made during development, such as latency vs. accuracy or cost vs. performance][cite: 1].

| **Tradeoff category** | **Selection** | **Justification/Evidence** |
|---|---|---|
| Latency vs Accuracy | Chosen Option | Measurement/Benchmark data |
| Tradeoff 2 | Chosen Option | Measurement/Benchmark data |
| Tradeoff 3 | Chosen Option | Measurement/Benchmark data |

Table 1: Summary of core engineering decisions

# 5 MLOps and evaluation

## 5.1 Automated pipelines

[Describe the automated pipeline for training, evaluation, and deployment/promotion][cite: 1].

## 5.2 Experiment tracking

[Document the use of MLflow, Weights & Biases, or equivalent for metric tracking][cite: 1].

## 5.3 Quality assurance and testing

[Detail the unit, integration, and end-to-end tests. Explain your strategy for regression protection and LLM evaluation][cite: 1].

# 6 Observability and security

## 6.1 Monitoring dashboard

[Describe the Prometheus/Grafana setup and the p50/p95 latency metrics][cite: 1].

### 6.2 Security and robustness

[Document secrets management, rate limiting, and failure mode behaviors like retries or fallbacks][cite: 1].

## 7 Cloud deployment and cost analysis

[Provide the public API endpoint URL and a detailed cost estimate including primary cost drivers][cite: 1].